

Functions in Python

1. What is a Function?

A **function** is a named block of code that performs a specific task. Once defined, it can be reused multiple times in a program. Functions are an important part of the "divide and conquer" strategy in programming: rather than writing a large program in one block, we break it into smaller, manageable parts called functions.

Key Characteristics of Functions

- A function can take input values (called arguments or parameters).
 - A function can perform operations using these inputs.
 - A function can optionally return an output value using the return statement.
 - A function may or may not have parameters, and may or may not return values.
-

2. Why Do We Need Functions?

1. **Reusability:** Write once, use multiple times.
 2. **Modularity:** Large programs can be broken into smaller, independent parts.
 3. **Readability:** Easier to understand and maintain.
 4. **Debugging:** Easy to test smaller components separately.
 5. **Abstraction:** Users of a function do not need to know the internal logic, only how to call it.
-

3. Defining and Calling Functions

Syntax

```
def function_name(parameters):  
    """Optional documentation string"""  
    # body of the function  
    return value # optional
```

Example 1: A Simple Function

```
def greet():  
    """This function prints a greeting message."""  
    print("Hello, welcome to Python functions.")
```

```
# function call
```

```
greet()
```

Output:

Hello, welcome to Python functions.

Here:

- `def` is the keyword used to define a function.
 - `greet` is the function name.
 - No parameters are passed.
 - The function simply prints a statement.
-

4. Types of Functions

a) Built-in Functions

These are already provided by Python. Examples: `print()`, `len()`, `sum()`, `range()`, `max()`, `min()`.

b) User-defined Functions

These are functions that we create using the `def` keyword.

5. Function Parameters and Arguments

Parameters are variables listed in the function definition. Arguments are the actual values passed when calling a function.

Example 2: Function with Parameters

```
def greet_user(name):  
    print("Hello", name)
```

```
greet_user("Atharva")
```

Output:

Hello Atharva

Types of Parameters

1. Positional Arguments

Arguments are matched with parameters in the order they are passed.

```
def add(a, b):
```

```
return a + b
```

```
print(add(5, 10)) # 15
```

2. Default Arguments

Provide default values to parameters. If the user does not supply a value, the default will be used.

```
def greet(name="Guest"):
    print("Hello", name)
```

```
greet()      # Hello Guest
```

```
greet("Atharva") # Hello Atharva
```

3. Keyword Arguments

Arguments can be passed by explicitly naming the parameter.

```
def student_info(name, age):
    print("Name:", name, " Age:", age)
```

```
student_info(age=20, name="Riya")
```

4. Variable-length Arguments

- `*args` → allows passing multiple positional arguments. They are stored as a tuple.
- `**kwargs` → allows passing multiple keyword arguments. They are stored as a dictionary.

```
def demo_args(*args):
    print(args)
```

```
demo_args(1, 2, 3, 4)
```

```
# Output: (1, 2, 3, 4)
```

```
def demo_kwargs(**kwargs):
    print(kwargs)
```

```
demo_kwargs(name="John", age=25)
```

```
# Output: {'name': 'John', 'age': 25}
```

6. Return Statement

A function can return values using the return keyword.

```
def square(num):  
    return num * num
```

```
result = square(6)
```

```
print(result) # 36
```

Notes:

- If no return is written, Python automatically returns None.
- Functions can return multiple values (as tuples).

```
def calculate(a, b):  
    return a+b, a-b, a*b
```

```
x, y, z = calculate(10, 5)
```

```
print(x, y, z) # 15 5 50
```

7. Scope of Variables

Scope means the region of code where a variable is accessible.

1. **Local Variable:** Defined inside a function, accessible only within that function.
2. **Global Variable:** Defined outside all functions, accessible throughout the program.

```
x = 50 # global variable
```

```
def demo():  
    x = 10 # local variable  
    print("Inside function:", x)
```

```
demo()
```

```
print("Outside function:", x)
```

Output:

Inside function: 10

Outside function: 50

To modify a global variable inside a function, use the global keyword.

```
x = 5
```

```
def change():
```

```
    global x
```

```
    x = 20
```

```
change()
```

```
print(x) # 20
```

8. Docstrings in Functions

A docstring is a string placed at the beginning of a function to describe what the function does. It is written in triple quotes.

```
def multiply(a, b):
```

```
    """This function multiplies two numbers."""
```

```
    return a * b
```

```
print(multiply.__doc__)
```

Output:

This function multiplies two numbers.

9. Lambda Functions (Anonymous Functions)

A lambda function is a small, one-line function without using def.

```
square = lambda x: x * x
```

```
print(square(7)) # 49
```

```
add = lambda a, b: a + b
```

```
print(add(3, 5)) # 8
```

Lambda functions are often used with `map()`, `filter()`, and `reduce()` for concise code.

10. Recursion Function

Excellent starting point. You've introduced recursion, factorial, and Fibonacci, which are the classic examples for teaching recursion. Let's expand your material into a **complete, in-depth explanation of recursion** that you can directly use with students. I'll refine the Fibonacci code too (your indentation in the snippet needs slight adjustment).

Recursive Functions in Python

What is Recursion?

A **recursive function** is a function that calls itself in order to solve a problem. Instead of solving the problem directly, the function breaks it into smaller sub-problems of the same type and solves those until reaching a **base case**.

A recursive function has two essential parts:

1. **Base Case** – the condition where recursion stops. Without this, the function would keep calling itself infinitely and cause an error.
 2. **Recursive Case** – the part of the function where it calls itself with a smaller/simpler input, moving closer to the base case.
-

Syntax of a Recursive Function

```
def function_name(parameters):  
    if base_case_condition:  
        return base_case_value  
    else:  
        return function_name(smaller_input)
```

Example 1: Factorial using Recursion

Factorial of a number n is defined as:

$$n! = n * (n-1) * (n-2) * \dots * 1$$

Special cases:

- $0! = 1$
- $1! = 1$

Code:

```
def factorial(n):  
    if n == 0 or n == 1: # base case  
        return 1  
    else:                # recursive case  
        return n * factorial(n - 1)
```

```
print(factorial(5)) # 120
```

Step-by-step Execution:

```
factorial(5)  
= 5 * factorial(4)  
= 5 * (4 * factorial(3))  
= 5 * (4 * (3 * factorial(2)))  
= 5 * (4 * (3 * (2 * factorial(1))))  
= 5 * (4 * (3 * (2 * 1)))  
= 120
```

Example 2: Fibonacci Series using Recursion

The Fibonacci sequence is defined as:

$F(0) = 0$

$F(1) = 1$

$F(n) = F(n-1) + F(n-2)$ for $n > 1$

Code:

```
def fibonacci(n):  
    if n == 0: # base case 1  
        return 0  
    elif n == 1: # base case 2  
        return 1  
    else:        # recursive case  
        return fibonacci(n-1) + fibonacci(n-2)
```

```
print(fibonacci(6)) # Output: 8
```

Step-by-step Execution (for n=6):

fibonacci(6)

= fibonacci(5) + fibonacci(4)

fibonacci(5) = fibonacci(4) + fibonacci(3)

fibonacci(4) = fibonacci(3) + fibonacci(2)

fibonacci(3) = fibonacci(2) + fibonacci(1)

...

until base cases are reached

Result:

0, 1, 1, 2, 3, 5, 8

So, fibonacci(6) = 8.

Key Points About Recursion

1. **Every recursive function must have a base case**; otherwise, it will result in **infinite recursion** and cause a `RecursionError`.
 2. Recursion can sometimes be **less efficient** than iterative solutions (e.g., Fibonacci) because of repeated calculations.
 3. Recursion is very powerful for problems that can be broken into smaller sub-problems like:
 - Factorials
 - Fibonacci numbers
 - Sum of digits
 - Tower of Hanoi
 - Binary search
 - Tree and graph traversals
-